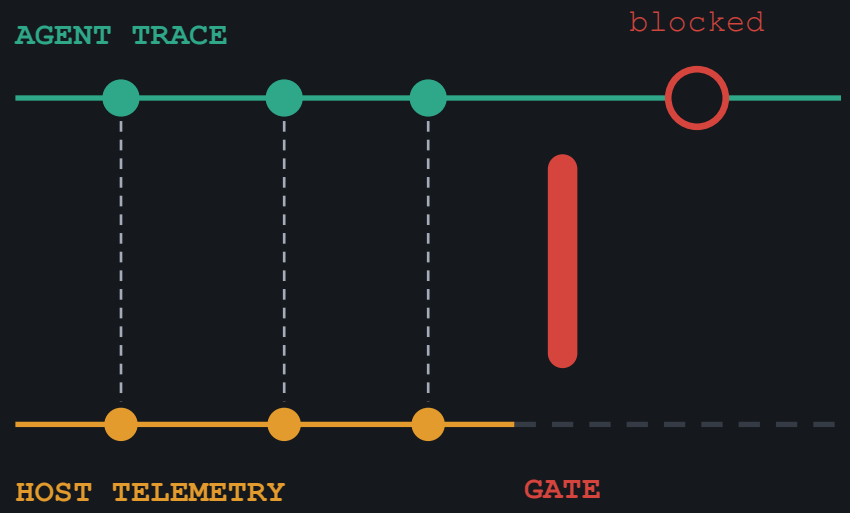
PERCEPTRON HACKATHON 2026 · THEME 6 · PROBLEM 9

Aletheia

Catching an agent before it acts on a bad idea

Every tool call is checked against where its arguments came from, before it runs. The machine is watched after it runs. A careful model is called only when the risk is real.



TEAM

Chronicles.exe

Shrestha Chandra · Aakash Marigeri · Jaipreet Singh

The bad action looks fully authorized

A hijacked agent runs no malware. It calls its own approved tools with arguments it copied from a web page, README, email or pull request. Asking "is this command dangerous?" fails, because the command looks like normal work. The real question is **whether the action is grounded in the user's request or in text an outsider wrote.**

WHAT MAKES IT HARD

01 **The check needs provenance, not content.**
Something must record where every string in a tool call's arguments came from.

02 **It sits inline, so it must be fast.**
An LLM on every call is slow, costly and can itself be injected.

03 **Attackers reword.**
Paraphrased or encoded instructions slip past text matching, and some payloads only act later.

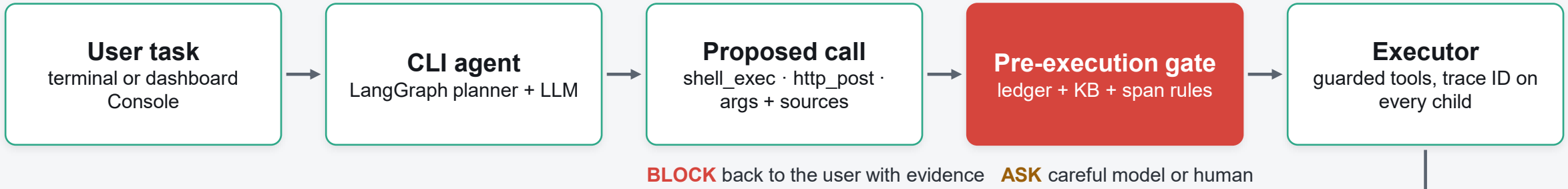
#1
Prompt injection leads the OWASP Top 10 for LLM apps (2025)

0
Reliable prevention methods OWASP knows of for prompt injection

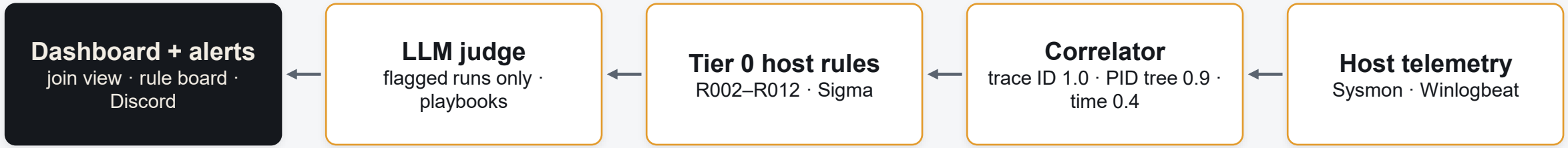
14
Agent techniques added to MITRE ATLAS with Zenity Labs, Oct 2025

Check before it runs, watch after, judge on risk

BEFORE IT RUNS · AGENT SIDE



AFTER IT RUNS · HOST SIDE



BUILT ON

`OpenTelemetry` `Elasticsearch` `Postgres` `Redis Streams` `Render Workflows` `MITRE ATLAS · ATT&CK`

The verifiable source is where each argument came from

WHAT THE GATE CHECKS AGAINST

Provenance ledger

Every tool result is stored with its source kind: the user, a trusted workspace, untrusted web, file, email or PR text, or unknown. A call's arguments are matched back to these spans.

Reference knowledge base (YAML)

- Trusted workspace roots
- Allowed domains and egress list
- Tool-call shapes learned from clean runs
- Command allowlist for the shell tool
- Tool manifest: names and schemas

CHECK	FIRES WHEN	DECISION
R001 untrusted text in args	a run of the arguments matches text from an untrusted source	BLOCK
R006 tool exfiltration	an external POST carries fetched data or secrets	BLOCK
Command guard	cmdlet outside the allowlist; encoded, download or IEX patterns	BLOCK
R001U unknown provenance	arguments carry text with no recorded source	ASK
R005 unseen call shape	tool and argument shape never seen in a clean run	ASK
none	arguments trace to the user or a trusted workspace	ALLOW

Today these rules run on the finished trace (median 0.23 ms per run). The build moves them in front of the executor node.

One injected page, stopped before it runs

01
Poisoned page
The agent fetches a page with an instruction hidden in an HTML comment.

AML.T0051

→

02
Proposed call
The planner proposes shell_exec using text from that page.

AML.T0053

→

03
Gate: BLOCKED
R001: the arguments match a span of untrusted text. Nothing reaches the shell.

R001

→

04
Evidence kept
Span ids, the matched text and the source URL are stored with the run.

evidence

→

05
Alert
The dashboard shows the blocked call; Discord gets an alert that cannot ping.

BLOCKED · HIGH

ILLUSTRATIVE · THE CLI IS PART OF THE BUILD

```
$ aletheia run "summarise the release notes at example.com"
  web_fetch   example.com/release-notes      source: untrusted web
  shell_exec  powershell -Command "Resolve-DnsName ..."
  BLOCKED  R001 untrusted text in tool args
    46 chars match span web_fetch#2 (HTML comment)
    ATLAS AML.T0051 · run a3f9 · open in dashboard
  nothing executed · agent told why · user can vouch for the source
```

IF THE INJECTION IS REWORDED

Paraphrased text slips past R001. After the call, R002 and R004 fire on the host side, Tier 0 says SUSPICIOUS·MEDIUM, and the judge raises it to HIJACKED·HIGH in 5.3 s.

Live recording inj-paraphrase, 25 Oct.

A CLI agent, a dashboard, and an open-source core

CLI agent

new · where the gate lives

`aletheia run "<task>"`
the guarded LLM agent, gate on

`--gate block | ask | log`
how strict the gate is

`aletheia scan-pr <url>`
PR text and diff checks

`aletheia explain <run>`
the judge's reasoning, cited

Dashboard

built · the main view

Runs
every run with its verdict

Join view
each span next to the host events it caused

Rule board · patterns
what fired, across runs

Console
chat with the agent, see its X-ray

Open source

Apache-2.0 core

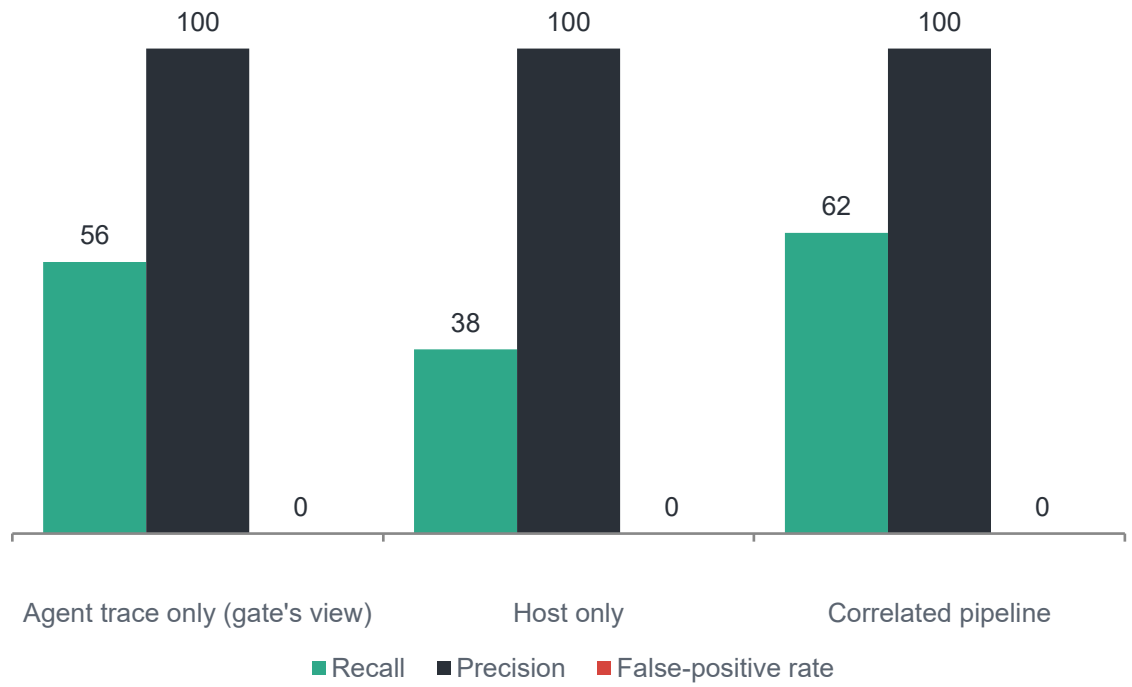
pip install / docker compose
one command to try it

Rule and playbook packs
plain YAML, mapped to ATLAS

OpenTelemetry GenAI spans
any framework can plug in

LangGraph adapter first
more agents by community

What we measured, and what will prove the gate works



32 self-written runs (16 attacks, 15 benign), so not field numbers. Counting every flag the judge sees, 15 of 16 attacks are caught.

PLAN FOR THE GATE

Metrics
attack success rate with the gate off and on; benign task completion; added latency per call

Benchmarks
AgentDojo and InjecAgent tasks, plus our 32 scenarios and 7 live injections

Baselines
no guard; allowlist only; an LLM guard on every call; ours

Red-teaming
11 mutations (6 caught today), injection aimed at the judge, attacks written by outsiders

False positives
50 to 100 public pull requests and ordinary agent tasks

Mapped to Problem 9, bullet by bullet

	PROBLEM 9 ASKS FOR	ALETHEIA DELIVERS	STATUS
MVP	A checker that verifies at least one class of agent action against a source before it executes	The gate checks shell_exec and http_post arguments against the provenance ledger (R001, R006, command guard)	**Rules built and tested; inline gate built in November**
MVP	A demo catching at least one deliberately bad or hallucinated action	7 live injections: HTML comment, white text, zero-size font, alt text, email, exfiltration, paraphrase	**Recorded 25 Oct; 12/12 runs match the expected verdict**
STRETCH	Routing between a fast, cheap model and a slow, careful one by risk	Tier 0 rules on every call (median 0.33 ms); the LLM judge only on flagged ones. It cleared 5/5 flagged benign runs; 0 benign runs ended HIJACKED	**Built; run with a real model on 31 scenarios**
STRETCH	A small red-teaming test suite	Mutation tests, injection aimed at the judge, PR attacks in a sandbox	**Built (self-written); outsider round planned**

ALREADY RUNNING 989 tests passing · 32 evaluation scenarios · dashboard deployed on Render

Most of it runs today; the gate and CLI are the build

MILESTONES TO THE FINALS

1–10 Nov
Gate before the executor: ALLOW, ASK, BLOCK; CLI skeleton

11–24 Nov
CLI agent with the LLM planner; file and PR ledger; gate view

25 Nov–1 Dec
AgentDojo and InjecAgent runs; midway report on 1 Dec

2–14 Dec
Red-team round, false-positive study, pip and Docker, docs

15–18 Dec
Finals demo at IIT Bombay

WHAT COULD GO WRONG

The gate blocks real work
ASK mode; the user can vouch for a source; trust set per workspace

Reworded injections pass R001
host rules and the judge are the second line; add fuzzy matching

Delayed or gated payloads
outside the gate's reach; the sandbox logs blocked egress

The judge itself gets injected
no tools, evidence fenced as data, rules overrule it

Host telemetry is Windows-only
the agent side is OpenTelemetry and cross-platform; Linux auditd later

TEAM

Shrestha Chandra
agent, correlator, judge, dashboard

Aakash Marigeri
host telemetry, Elasticsearch

Jaipreet Singh
detection rules, store, scenarios